

Reviewer Pack — Minimal Rank of Hypergeometric Sequences Bounds Resolution P...

Entry 6f03a4ee1c85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 02:50:13 UTC

Minimal Rank of Hypergeometric Sequences Bounds Resolution Proof Depth

Entry ID: 6f03a4ee1c85 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 02:50:13 UTC

1. Conjecture

Field A (mathematical branch): Hypergeometric Sequences **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{‘sentence_1’: ‘For any given instance of a CNF formula, the minimal rank of its corresponding hypergeometric sequence is at least $\Omega(\log n)$ ’, ‘sentence_2’: “where ‘rank’ refers to the dimension of the vector space spanned by the elements of the sequence.”, ‘sentence_3’: ‘This lower bound holds for all instances with resolution proof depth d , implying that $d \geq \Omega(\log n / \log \log n)$.’}

Rationale (proposer’s reasoning):

{‘sentence_1’: ‘Hypergeometric sequences often appear in combinatorial problems and have connections to algebraic geometry.’, ‘sentence_2’: ‘The conjecture suggests that properties of hypergeometric sequences could be used to analyze the structure of resolution proofs, potentially leading to new insights into complexity theory.’, ‘sentence_3’: ‘If true, this would provide a novel approach to understanding the complexity of Resolution Proof Complexity.’}

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 575998e2bc0480ab

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all tested CNF formulas with n variables and resolution proof depth d , the minimal rank of the hypergeometric sequence is at least $\Omega(\log n)$ AND the ratio of the mean rank to the number of variables ($\text{mean_rank}/n$) is greater than or equal to 0.8.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "hypergeometric sequences" AND "resolution proof complexity" - "minimal rank" AND hypergeometric AND CNF formula" - "resolution proof depth" AND lower bound AND hypergeometric sequence

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i + max(range(i, rows), key=lambda x: abs(matrix[x][i]))
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            pivot = matrix[i][i]
            if pivot == 0:
                continue
            for j in range(cols):
                matrix[i][j] /= pivot
            for k in range(rows):
                if k != i:
                    factor = matrix[k][i]
                    for j in range(cols):
                        matrix[k][j] -= factor * matrix[i][j]
        return matrix

    def rank(matrix):
        rows, cols = len(matrix), len(matrix[0])
        row echelon_form = gaussian_elimination(matrix)
        rank = 0
        for i in range(rows):
            if any(row[i] != 0 for row in row echelon_form):
```

```

        rank += 1
    return rank

def resolution_proof_depth(cnf_formula):
    # Simplified DPLL solver to estimate proof depth
    stack = []
    assignment = {}
    def dpll():
        if not cnf_formula:
            return True, 0
        literal = next((lit for lit in cnf_formula[0] if lit not in assignment and -lit not in assignment))
        if literal is None:
            return False, 0
        assignment[literal] = True
        stack.append(literal)
        depth = 1
        while stack:
            lit = stack[-1]
            if any(all(assignment[var] == (var > 0) for var in clause) for clause in cnf_formula[0]):
                return True, depth
            neg_lit = -lit
            if neg_lit in assignment and not assignment[neg_lit]:
                stack.pop()
                del assignment[lit]
                continue
            new_clause = [var for var in cnf_formula[0] if var != lit and -var not in assignment]
            cnf_formula[0] = new_clause
            depth += 1
        return False, 0
    return dpll()[1]

def generate_cnf(n):
    clauses = []
    for i in range(2**n):
        clause = [random.choice([-1, 1]) * (j + 1) for j in range(n)]
        if any(all(clause[var] == (var > 0) for var in clause) for clause in clauses):
            continue
        clauses.append(clause)
    return clauses

n_values = [5, 10, 15, 20, 30, 40]
total_rank = 0
instances_tested = 0
conjecture_holds = True
counterexample = ""

for n in n_values:
    cnf_formula = generate_cnf(n)
    proof_depth = resolution_proof_depth(cnf_formula)
    if proof_depth == 0:
        continue
    hypergeometric_sequence = [len([var for var in clause if var > 0]) for clause in cnf_formula]
    rank_value = rank(hypergeometric_sequence)
    total_rank += rank_value
    instances_tested += 1

```

```

    if rank_value < math.log(n):
        conjecture_holds = False
        counterexample = f"n={n}, proof_depth={proof_depth}, rank={rank_value}"

mean_rank = Fraction(total_rank, instances_tested) / n
support_fraction = instances_tested / len(n_values)

return {
    "metric_name": "mean_rank/n",
    "metric_value": float(mean_rank),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_rank = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_rank} std=0.0 support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture based on the results. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11283
2	preregistration	ollama_remote	glm4:latest	0	0	5668
3	novelty	ollama_remote	glm4:latest	0	0	4846
4	novelty	ollama_remote	glm4:latest	0	0	5486
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17397
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21991
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13730
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13695
9	judge	ollama_remote	glm4:latest	0	0	45693

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 139788 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/6f03a4ee1c85.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/6f03a4ee1c85.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/6f03a4ee1c85.tar.gz* (if generated)